🚀 De Zéro (Auguste) à Héros (JM): Apprendre la Programmation avec C++ et C#

Module 1 : Le "Hello World" et la structure d'un programme

La tradition veut que le tout premier programme que l'on écrive affiche simplement "Hello World!" (Bonjour le monde !) à l'écran. Cela permet de comprendre le "squelette" obligatoire d'un programme.

- Le point de départ de tout programme s'appelle la fonction `main` (en C++) ou `Main` (en C#). C'est la porte d'entrée.
- Presque chaque instruction se termine par un **point-virgule** `;`. C'est le point final de ta phrase pour l'ordinateur.
- Les **accolades** `{ }` servent à délimiter des blocs de code (un début et une fin).
- Les lignes qui commencent par `//` sont des **commentaires** : l'ordinateur les ignore totalement, ce sont des notes pour toi !

En C++

```
// On importe la boîte à outils "iostream" qui permet d'afficher du texte à l'écran

#include <iostream>

// Point d'entrée du programme

int main() {

    // std::cout envoie du texte vers l'écran. std::endl fait un retour à la ligne.

    std::cout << "Hello World!" << std::endl;

    // Indique à l'ordinateur que le programme s'est terminé sans erreur (code 0)

    return 0;

}
```

En C#

```
// On utilise la boîte à outils "System" de base

using System;

// En C#, tout le code doit obligatoirement être rangé dans une "classe" (on verra ça au Module 6 !)

class Program

{

    // Point d'entrée du programme (avec un 'M' majuscule en C#)

    static void Main()

    {

        // Console.WriteLine écrit une ligne de texte à l'écran et va à la ligne automatiquement

        Console.WriteLine("Hello World!");

    }

}
```

Module 2 : Les Variables et les Interactions

Une **variable**, c'est une boîte rangée dans la mémoire de l'ordinateur. On colle une **étiquette** dessus (son nom) et on met une **valeur** à l'intérieur.

En C++ et C#, on doit préciser la taille et la forme de la boîte avant de l'utiliser : c'est ce qu'on appelle le **Type**.

- `int` (integer) : Un nombre entier (ex: `42`, `-5`).
- `double` ou `float` : Un nombre à virgule (ex: `19.99` - *attention, en code on utilise un point `.` et non une virgule*).
- `bool` (booléen) : Un interrupteur qui ne peut valoir que Vrai (`true`) ou Faux (`false`).

- `string` : Une chaîne de caractères, c'est-à-dire du texte, toujours entre guillemets `" "` (ex: `"Arthur"`).

Exemple : Créer des boîtes et interagir avec l'utilisateur

En C++

```
#include <iostream>

#include <string> // Nécessaire en C++ pour utiliser le type "string"

int main() {

    // 1. Création et remplissage des variables

    std::string prenom = "Alex";

    int age = 25;

    double taille = 1.75;

    bool estDebutant = true;

    // 2. Affichage des variables en les combinant avec des chevrons <<

    std::cout << "Bonjour, je suis " << prenom << " et j'ai " << age << " ans." << std::endl;

    // 3. Demander une information au joueur

    int anneeNaissance; // On crée une boîte vide

    std::cout << "En quelle annee es-tu ne ? ";

    std::cin >> anneeNaissance; // std::cin récupère ce que l'utilisateur tape au clavier

    // 4. Petit calcul avec + , - , * (multiplier), / (diviser)

    int ageCalcule = 2026 - anneeNaissance;

    std::cout << "Tu as donc environ " << ageCalcule << " ans !" << std::endl;

    return 0;
```

```
}
```

En C#

```
using System;

class Program

{

    static void Main()

    {

        // 1. Création et remplissage des variables

        string prenom = "Alex";

        int age = 25;

        double taille = 1.75;

        bool estDebutant = true;

        // 2. Affichage : en C#, le symbole $ avant les guillemets permet d'insérer

        // directement des variables dans le texte avec des accolades { } !

        Console.WriteLine($"Bonjour, je suis {prenom} et j'ai {age} ans.");

        // 3. Demander une information au joueur

        Console.Write("En quelle annee es-tu ne ? ");

        string saisieClavier = Console.ReadLine(); // ReadLine récupère toujours du texte (string)

        // On convertit ce texte en nombre entier (int) avec int.Parse()

        int anneeNaissance = int.Parse(saisieClavier);

        // 4. Petit calcul
```

```
        int ageCalcule = 2026 - anneeNaissance;

        Console.WriteLine($"Tu as donc environ {ageCalcule} ans !");

    }

}
```

Module 3 : Prendre des décisions (Les Conditions)

Un programme intelligent doit pouvoir s'adapter. Pour cela, on utilise des conditions : **"SI telle chose est vraie, ALORS fais ceci, SINON fais cela"**.

Les outils de comparaison :

- `==` : Est strictement égal à *(Attention : un seul `=` range une valeur dans une boîte, deux `==` comparent deux valeurs !)*
- `!=` : Est différent de
- `>` / `<` : Plus grand que / Plus petit que
- `>=` / `<=` : Plus grand ou égal / Plus petit ou égal
- `&&` : ET (les deux conditions doivent être vraies)
- `||` : OU (au moins l'une des deux doit être vraie)

1. Le bloc `if / else if / else` (Si / Sinon si / Sinon)

En C++

```
#include <iostream>

int main() {

    int pointsDeVie = 35;

    if (pointsDeVie <= 0) {

        std::cout << "Game Over ! Tu es mort." << std::endl;
```

```
    }
    else if (pointsDeVie < 50) {
        std::cout << "Attention, ta sante est basse ! Prends une potion." << std::endl;
    }
    else {
        std::cout << "Tout va bien, tu es en pleine forme !" << std::endl;
    }
    return 0;
}
```

En C#

```
using System;
class Program
{
    static void Main()
    {
        int pointsDeVie = 35;
        if (pointsDeVie <= 0)
        {
            Console.WriteLine("Game Over ! Tu es mort.");
        }
        else if (pointsDeVie < 50)
```

```
        {
            Console.WriteLine("Attention, ta sante est basse ! Prends une potion.");
        }
        else
        {
            Console.WriteLine("Tout va bien, tu es en pleine forme !");
        }
    }
}
```

2. Le `switch` (L'aiguillage pour les menus)

Quand on veut tester la valeur exacte d'une seule variable (comme le choix d'un joueur dans un menu), le `switch` est beaucoup plus propre qu'une longue liste de `if`.

En C++

```
#include <iostream>

int main() {
    int choixMenu = 2;
    switch (choixMenu) {
        case 1:
            std::cout << "Lancement d'une nouvelle partie..." << std::endl;
            break; // "break" est obligatoire pour sortir du switch !
        case 2:
            std::cout << "Chargement de la sauvegarde..." << std::endl;
```

```
            break;
        case 3:
            std::cout << "Fermeture du jeu." << std::endl;
            break;
        default: // Si le nombre n'est ni 1, ni 2, ni 3
            std::cout << "Choix invalide !" << std::endl;
            break;
    }
    return 0;
}
```

En C#

```
using System;
class Program
{
    static void Main()
    {
        int choixMenu = 2;
        switch (choixMenu)
        {
            case 1:
                Console.WriteLine("Lancement d'une nouvelle partie...");
```

```
            break;
        case 2:
            Console.WriteLine("Chargement de la sauvegarde...");
            break;
        case 3:
            Console.WriteLine("Fermeture du jeu.");
            break;
        default:
            Console.WriteLine("Choix invalide !");
            break;
    }
  }
}
```

Module 4 : Répéter des actions (Les Boucles)

Si tu dois afficher 100 fois un message ou faire combattre un joueur jusqu'à ce qu'un monstre meure, tu ne vas pas copier-coller ton code 100 fois. Tu vas utiliser une **boucle**. Il en existe 4 types !

1. La boucle `while` (Tant que...)

Elle vérifie la condition **avant** de commencer. Tant que la condition est vraie, elle tourne en rond.

En C++

```
#include <iostream>

int main() {
```

```
    int energie = 3;
    while (energie > 0) {
        std::cout << "Je donne un coup d'epee ! Energie restante : " << energie << std::endl;
        energie = energie - 1; // On diminue l'énergie, sinon la boucle ne s'arrête jamais !
    }
    std::cout << "Je suis epuise." << std::endl;
    return 0;
}
```

En C#

```
using System;
class Program
{
    static void Main()
    {
        int energie = 3;
        while (energie > 0)
        {
            Console.WriteLine($"Je donne un coup d'epee ! Energie restante : {energie}");
            energie = energie - 1;
        }
        Console.WriteLine("Je suis epuise.");
```

```
    }
}
```

2. La boucle `do...while` (Faire... tant que)

À l'inverse de `while`, elle exécute le code **au moins une fois**, puis vérifie la condition à la fin. Très pratique pour redemander une saisie à l'utilisateur tant qu'il se trompe !

En C++

```
#include <iostream>

int main() {

    int codeSecret;

    do {

        std::cout << "Entrez le code secret (1234) : ";

        std::cin >> codeSecret;

    } while (codeSecret != 1234); // Recommence TANT QUE le code est différent de 1234

    std::cout << "Coffre ouvert !" << std::endl;

    return 0;

}
```

En C#

```
using System;

class Program

{

    static void Main()

    {
```

```
        int codeSecret;

        do

        {

            Console.Write("Entrez le code secret (1234) : ");

            codeSecret = int.Parse(Console.ReadLine());

        } while (codeSecret != 1234);

        Console.WriteLine("Coffre ouvert !");

    }

}
```

3. La boucle `for` (Pour un compteur allant de X à Y...)

C'est la boucle idéale quand **on connaît d'avance le nombre de répétitions**. Elle rassemble sur une seule ligne : la création du compteur (`int i = 1`), la condition d'arrêt (`i <= 5`), et l'augmentation du compteur à chaque tour (`i++`, qui est un raccourci pour écrire `i = i + 1`).

En C++

```
#include <iostream>

int main() {

    // Compte de 1 jusqu'à 5 inclus

    for (int i = 1; i <= 5; i++) {

        std::cout << "Tour de piste numero " << i << std::endl;

    }

    return 0;

}
```

En C#

```
using System;

class Program

{

    static void Main()

    {

        for (int i = 1; i <= 5; i++)

        {

            Console.WriteLine($"Tour de piste numero {i}");

        }

    }

}
```

4. Les Tableaux et la boucle `foreach` (Pour chaque élément...)

Un **tableau** (ou *Array*) est une grande boîte divisée en plusieurs cases, permettant de stocker une liste d'éléments (par exemple, un inventaire). La boucle `foreach` (ou `for(:)` en C++) est magique : elle parcourt automatiquement chaque case du tableau du début à la fin, sans avoir besoin de compter !

En C++

```
#include <iostream>

#include <string>

int main() {

    // Création d'un tableau de 3 textes

    std::string inventaire[3] = {"Epee", "Bouclier", "Potion"};
```

```
    std::cout << "Contenu du sac a dos :" << std::endl;
    // Pour chaque "objet" présent dans "inventaire"
    for (std::string objet : inventaire) {
        std::cout << "- " << objet << std::endl;
    }
    return 0;
}
```

En C#

```
using System;
class Program
{
    static void Main()
    {
        // Création d'un tableau de textes en C#
        string[] inventaire = { "Epee", "Bouclier", "Potion" };
        Console.WriteLine("Contenu du sac a dos :");
        // En C#, le mot-clé s'appelle explicitement "foreach"
        foreach (string objet in inventaire)
        {
            Console.WriteLine($"- {objet}");
        }
```

```
    }
}
```

Module 5 : Les Fonctions (Sous-programmes)

Imagine que dans ton programme, tu doives calculer des dégâts d'attaque à 10 endroits différents. Pour éviter de réécrire la formule 10 fois, on crée une **Fonction** (aussi appelée *Méthode* en C#).

Une fonction est comme un **mixeur de cuisine** :

1. Tu lui donnes des ingrédients en entrée (les **paramètres**).
2. Il fait son travail à l'intérieur.
3. Il te rend le smoothie à la sortie (le **type de retour** avec le mot-clé `return`). *(Note : si la fonction fait juste une action comme afficher du texte sans rien renvoyer, son type de retour est* ***`void`****, qui veut dire "vide").*

En C++

```
#include <iostream>

// 1. Fonction qui ne renvoie rien (void)

void afficherBanniere() {

    std::cout << "=== SUPER JEU RPG ===" << std::endl;

}

// 2. Fonction qui prend deux entiers (int) et renvoie un entier (int)

int calculerDegats(int forceAttaquant, int armureDefenseur) {

    int resultat = forceAttaquant - armureDefenseur;

    if (resultat < 0) {

        resultat = 0; // On évite les dégâts négatifs !
```

```
    }
    return resultat; // On renvoie le résultat final
}
int main() {
    afficherBanniere(); // On appelle notre première fonction
    // On appelle la 2ème fonction et on range son résultat dans une variable
    int degatsSubis = calculerDegats(25, 10);
    std::cout << "Le monstre subit " << degatsSubis << " points de degats !" << std::endl;
    return 0;
}
```

En C#

```
using System;
class Program
{
    // 1. Fonction qui ne renvoie rien (void)
    static void AfficherBanniere()
    {
        Console.WriteLine("=== SUPER JEU RPG ===");
    }
    // 2. Fonction qui prend deux entiers et renvoie un entier
    static int CalculerDegats(int forceAttaquant, int armureDefenseur)
```

```
    {
        int resultat = forceAttaquant - armureDefenseur;
        if (resultat < 0)
        {
            resultat = 0;
        }
        return resultat;
    }
    static void Main()
    {
        AfficherBanniere();
        int degatsSubis = CalculerDegats(25, 10);
        Console.WriteLine($"Le monstre subit {degatsSubis} points de degats !");
    }
}
```

Module 6 : La Programmation Orientée Objet (POO)

C'est ici que la vraie magie opère ! Jusqu'à présent, nos variables et nos fonctions étaient séparées. Mais dans le monde réel (et dans les jeux vidéo), nous sommes entourés d'**Objets** (des voitures, des monstres, des joueurs).

1. Classe vs Objet : Le moule à gâteau

- Une **Classe** (`class`), c'est le **plan d'architecte** ou le **moule à gâteau**. Elle définit ce qu'est un Personnage en général :
    - Ses **Attributs** (ses variables internes : son `nom`, ses `pointsDeVie`, sa `force`).

    - Ses **Méthodes** (ses fonctions internes : `attaquer()`, `seSoigner()`).

- Un **Objet**, c'est le **gâteau concret** fabriqué à partir du moule. Avec *une seule* classe `Personnage`, tu peux créer 50 objets différents (`heros`, `gobelin1`, `bossFinal`) qui auront chacun leurs propres points de vie !

2. Le Constructeur et l'Encapsulation

- **L'Encapsulation (`private` vs `public`)** : Pour éviter qu'un bug dans le programme ne modifie accidentellement les points de vie d'un personnage à `-9999`, on protège les variables en les mettant en **`private`** (privé : utilisable uniquement à l'intérieur de la classe). Les actions autorisées (les méthodes) sont mises en **`public`** (utilisables par tout le monde).

- **Le Constructeur** : C'est une méthode spéciale qui porte **exactement le même nom que la classe**. Elle se lance automatiquement à la naissance de l'objet pour remplir ses variables de départ.

En C++

Note importante en C++ : Quand on passe un objet à une fonction pour qu'elle le modifie (par exemple, la cible d'une attaque qui doit perdre des PV), on ajoute le symbole **`&`** (passage par référence). Sans ce `&`, le C++ travaillerait sur une photocopie de la cible !

```
#include <iostream>

#include <string>

// Création du moule (la Classe)

class Personnage {

private:

    // Attributs privés (protégés à l'intérieur de la classe)

    std::string nom;

    int pointsDeVie;

    int force;

public:
```

```
    // Constructeur : appelé automatiquement à la création du personnage
    Personnage(std::string nomChoisi, int pvDepart, int forceDepart) {
        nom = nomChoisi;
        pointsDeVie = pvDepart;
        force = forceDepart;
    }
    // Méthode publique pour afficher l'état du personnage
    void afficherStatut() {
        std::cout << "[" << nom << "] PV : " << pointsDeVie << " | Force : " << force << std::endl;
    }
    // Méthode pour recevoir des dégâts (protège les PV pour ne pas descendre sous 0)
    void recevoirDegats(int nbDegats) {
        pointsDeVie = pointsDeVie - nbDegats;
        if (pointsDeVie < 0) {
            pointsDeVie = 0;
        }
        std::cout << nom << " perd " << nbDegats << " PV ! (Reste : " << pointsDeVie << " PV)" <<
std::endl;
    }
    // Méthode pour attaquer un AUTRE personnage (note le '&' pour modifier la vraie cible)
    void attaquer(Personnage& cible) {
        std::cout << nom << " attaque " << cible.nom << " !" << std::endl;
```

```
        cible.recevoirDegats(force);
    }
}; // Attention : en C++, il faut un point-virgule à la fin d'une classe !
int main() {
    // Création de 2 objets concrets à partir du moule Personnage
    Personnage heros("Arthur", 100, 25);
    Personnage monstre("Orque", 60, 15);
    heros.afficherStatut();
    monstre.afficherStatut();
    std::cout << "--- Debut du combat ---" << std::endl;
    heros.attaquer(monstre);
    monstre.attaquer(heros);
    return 0;
}
```

En C#

Note en C# : Les objets de classe sont automatiquement passés par référence (pas besoin de `&`), et pour créer un objet, on utilise le mot-clé **`new`**.

```
using System;
class Personnage
{
    // Attributs privés
```

```
private string nom;
private int pointsDeVie;
private int force;
// Constructeur
public Personnage(string nomChoisi, int pvDepart, int forceDepart)
{
    nom = nomChoisi;
    pointsDeVie = pvDepart;
    force = forceDepart;
}
// Méthodes publiques
public void AfficherStatut()
{
    Console.WriteLine($"[{nom}] PV : {pointsDeVie} | Force : {force}");
}
public void RecevoirDegats(int nbDegats)
{
    pointsDeVie = pointsDeVie - nbDegats;
    if (pointsDeVie < 0)
    {
        pointsDeVie = 0;
    }
```

```
        Console.WriteLine($"{nom} perd {nbDegats} PV ! (Reste : {pointsDeVie} PV)");
    }
    public void Attaquer(Personnage cible)
    {
        Console.WriteLine($"{nom} attaque {cible.nom} !");
        cible.RecevoirDegats(force);
    }
}
class Program
{
    static void Main()
    {
        // Création de 2 objets avec le mot-clé "new"
        Personnage heros = new Personnage("Arthur", 100, 25);
        Personnage monstre = new Personnage("Orque", 60, 15);
        heros.AfficherStatut();
        monstre.AfficherStatut();
        Console.WriteLine("--- Debut du combat ---");
        heros.Attaquer(monstre);
        monstre.Attaquer(heros);
    }
```

}

3. Bonus POO : L'Héritage (Créer des classes spécialisées)

Et si on voulait créer un `Magicien` ? Un magicien **est un** `Personnage` (il a un nom, des PV, de la force), mais il a en plus du `mana` et peut lancer un `sortDeFeu()`. Plutôt que de copier-coller toute la classe `Personnage`, on utilise **l'héritage** (symbole `:` dans les deux langages) ! La classe enfant (`Magicien`) hérite automatiquement de tout le code de la classe parent (`Personnage`). *(Pour que la classe enfant ait le droit d'accéder aux variables de son parent, on remplace `private` par **`protected`** dans le parent).*

En C++

```
#include <iostream>

#include <string>

class Personnage {

protected: // Accessible par cette classe ET ses enfants !

    std::string nom;

    int pointsDeVie;

public:

    Personnage(std::string n, int pv) : nom(n), pointsDeVie(pv) {}

    void saluer() {

        std::cout << "Je suis " << nom << " (" << pointsDeVie << " PV)." << std::endl;

    }

};

// Magicien hérite de Personnage

class Magicien : public Personnage {

private:
```

```
    int mana;
public:
    // Le constructeur du Magicien appelle celui de Personnage, puis initialise son mana
    Magicien(std::string n, int pv, int m) : Personnage(n, pv) {
        mana = m;
    }
    void lancerBouleDeFeu() {
        std::cout << nom << " lance une Boule de Feu ! (Mana restant : " << mana - 10 << ")" <<
std::endl;
        mana -= 10;
    }
};
int main() {
    Magicien gandalf("Gandalf", 80, 100);
    gandalf.saluer();            // Méthode héritée de Personnage !
    gandalf.lancerBouleDeFeu();  // Méthode propre au Magicien !
    return 0;
}
```

En C#

```
using System;
class Personnage
{
```

```
    protected string nom;
    protected int pointsDeVie;
    public Personnage(string n, int pv)
    {
        nom = n;
        pointsDeVie = pv;
    }
    public void Saluer()
    {
        Console.WriteLine($"Je suis {nom} ({pointsDeVie} PV).");
    }
}
// Magicien hérite de Personnage avec les deux-points ":"
class Magicien : Personnage
{
    private int mana;
    // ": base(n, pv)" envoie le nom et les PV au constructeur du parent (Personnage)
    public Magicien(string n, int pv, int m) : base(n, pv)
    {
        mana = m;
    }
```

```
    public void LancerBouleDeFeu()
    {
        mana -= 10;
        Console.WriteLine($"{nom} lance une Boule de Feu ! (Mana restant : {mana})");
    }
}
class Program
{
    static void Main()
    {
        Magicien gandalf = new Magicien("Gandalf", 80, 100);
        gandalf.Saluer();
        gandalf.LancerBouleDeFeu();
    }
}
```

🏆 Module 7 : Le Projet Final — "Les Chroniques de l'Arène"

Tu connais maintenant toutes les briques fondamentales de la programmation ! Il est temps de les assembler pour réaliser ton premier vrai jeu vidéo textuel : **un jeu d'arène au tour par tour**.

Cahier des charges du projet :

1. **POO :** Créer une classe `Combattant` (avec nom, PV, PV max, force, et nombre de potions).
2. **Tableaux & Boucle `for` :** Le joueur devra affronter une vague de **3 monstres successifs** stockés dans un tableau.

3. **Boucle `while` :** Tant que le héros et le monstre actuel sont en vie, le combat continue tour par tour.

4. **Boucle `do...while` & `switch` :** À chaque tour, afficher un menu d'actions (1. Attaquer, 2. Boire une potion) et redemander tant que le choix est invalide.

Prends le temps de lire le code ci-dessous ligne par ligne : tu es capable de tout comprendre !

Code complet du projet en C++

```
#include <iostream>

#include <string>

class Combattant {

private:

    std::string nom;

    int pv;

    int pvMax;

    int force;

    int potions;

public:

    // Constructeur

    Combattant(std::string nomChoisi, int pvDepart, int forceDepart, int nbPotions) {

        nom = nomChoisi;

        pv = pvDepart;

        pvMax = pvDepart;

        force = forceDepart;
```

```
        potions = nbPotions;
    }
    // Méthodes pour lire certaines informations privées (on les appelle des "Getters")
    std::string getNom() { return nom; }
    bool estVivant() { return pv > 0; }
    void afficherBarreDeVie() {
        std::cout << "[" << nom << "] PV : " << pv << "/" << pvMax
                  << " | Force : " << force << " | Potions : " << potions << std::endl;
    }
    void recevoirDegats(int degats) {
        pv -= degats;
        if (pv < 0) {
            pv = 0;
        }
        std::cout << "  -> " << nom << " subit " << degats << " degats ! (PV restants : " << pv << ")"
<< std::endl;
    }
    void attaquer(Combattant& cible) {
        std::cout << "\n⚔️ " << nom << " attaque " << cible.getNom() << " !" << std::endl;
        cible.recevoirDegats(force);
    }
    void boirePotion() {
```

```
        if (potions > 0) {

            potions--;

            int soin = 35;

            pv += soin;

            if (pv > pvMax) {

                pv = pvMax; // On ne dépasse pas les PV maximums

            }

            std::cout << "\n🧪 " << nom << " boit une potion et recupere " << soin

                    << " PV ! (PV : " << pv << "/" << pvMax << ", Potions restantes : " << potions <<
")" << std::endl;

        } else {

            std::cout << "\n❌ Plus de potions dans le sac ! Tu perds ton tour en cherchant..." <<
std::endl;

        }

    }

};

int main() {

    std::cout << "=========================================" << std::endl;

    std::cout << "   BIENVENUE DANS LES CHRONIQUES DE L'ARENE" << std::endl;

    std::cout << "=========================================\n" << std::endl;

    std::string nomJoueur;

    std::cout << "Entrez le nom de votre Heros : ";

    std::cin >> nomJoueur;
```

```
    // Création du héros : 100 PV, 25 de Force, 3 Potions
    Combattant heros(nomJoueur, 100, 25, 3);
    // Création d'un tableau de 3 ennemis de plus en plus forts !
    Combattant listeEnnemis[3] = {
        Combattant("Gobelin Eclaireur", 45, 12, 0),
        Combattant("Chevalier Squelette", 70, 18, 0),
        Combattant("Dragon Noir (BOSS)", 110, 24, 0)
    };
    // Boucle FOR pour affronter les 3 ennemis l'un après l'autre
    for (int i = 0; i < 3; i++) {
        Combattant& ennemiActuel = listeEnnemis[i];
        std::cout << "\n----------------------------------------" << std::endl;
        std::cout << "🔥 COMBAT " << (i + 1) << "/3 : Un " << ennemiActuel.getNom() << "
sauvage apparait !" << std::endl;
        std::cout << "----------------------------------------" << std::endl;
        // Boucle WHILE : tant que les deux combattants sont vivants
        while (heros.estVivant() && ennemiActuel.estVivant()) {
            std::cout << "\n--- Etat du combat ---" << std::endl;
            heros.afficherBarreDeVie();
            ennemiActuel.afficherBarreDeVie();
            int choixAction = 0;
            // Boucle DO...WHILE pour s'assurer que le joueur tape 1 ou 2
```

```
            do {
                std::cout << "\nQue veux-tu faire ? (1: Attaquer | 2: Boire une potion) : ";
                std::cin >> choixAction;
            } while (choixAction != 1 && choixAction != 2);
            // SWITCH pour exécuter l'action choisie par le joueur
            switch (choixAction) {
                case 1:
                    heros.attaquer(ennemiActuel);
                    break;
                case 2:
                    heros.boirePotion();
                    break;
            }
            // Riposte de l'ennemi s'il a survécu à l'attaque du joueur
            if (ennemiActuel.estVivant()) {
                ennemiActuel.attaquer(heros);
            } else {
                std::cout << "\n🏆 Victoire ! " << ennemiActuel.getNom() << " a ete terrasse !" <<
std::endl;
            }
        }
        // Si le héros est mort pendant ce combat, on arrête la boucle for avec "break"
```

```
        if (!heros.estVivant()) {

            std::cout << "\n💀 Tu es tombe au combat face a " << ennemiActuel.getNom() << "..."
<< std::endl;

            break;

        }

    }

    // Bilan final de la partie

    std::cout << "\n========================================" << std::endl;

    if (heros.estVivant()) {

        std::cout << "🎉 FELICITATIONS " << heros.getNom() << " ! Tu es le grand Champion de
l'Arene !" << std::endl;

    } else {

        std::cout << "   GAME OVER - Retente ta chance !" << std::endl;

    }

    std::cout << "========================================" << std::endl;

    return 0;

}
```

Code complet du projet en C#

```
using System;

class Combattant

{

    private string nom;
```

```
private int pv;

private int pvMax;

private int force;

private int potions;

// Constructeur

public Combattant(string nomChoisi, int pvDepart, int forceDepart, int nbPotions)

{

    nom = nomChoisi;

    pv = pvDepart;

    pvMax = pvDepart;

    force = forceDepart;

    potions = nbPotions;

}

public string GetNom() { return nom; }

public bool EstVivant() { return pv > 0; }

public void AfficherBarreDeVie()

{

    Console.WriteLine($"[{nom}] PV : {pv}/{pvMax} | Force : {force} | Potions : {potions}");

}

public void RecevoirDegats(int degats)

{
```

```
    pv -= degats;
    if (pv < 0)
    {
        pv = 0;
    }
    Console.WriteLine($"  -> {nom} subit {degats} degats ! (PV restants : {pv})");
}
public void Attaquer(Combattant cible)
{
    Console.WriteLine($"\n⚔️ {nom} attaque {cible.GetNom()} !");
    cible.RecevoirDegats(force);
}
public void BoirePotion()
{
    if (potions > 0)
    {
        potions--;
        int soin = 35;
        pv += soin;
        if (pv > pvMax)
        {
            pv = pvMax;
```

```
            }

            Console.WriteLine($"\n🧪 {nom} boit une potion et recupere {soin} PV ! (PV : {pv}/{pvMax}, Potions restantes : {potions})");

        }

        else

        {

            Console.WriteLine("\n❌ Plus de potions dans le sac ! Tu perds ton tour en cherchant...");

        }

    }

}

class Program

{

    static void Main()

    {

        Console.WriteLine("========================================");

        Console.WriteLine("   BIENVENUE DANS LES CHRONIQUES DE L'ARENE");

        Console.WriteLine("========================================\n");

        Console.Write("Entrez le nom de votre Heros : ");

        string nomJoueur = Console.ReadLine();

        // Création du héros

        Combattant heros = new Combattant(nomJoueur, 100, 25, 3);
```

```
        // Création du tableau d'ennemis
        Combattant[] listeEnnemis = {
            new Combattant("Gobelin Eclaireur", 45, 12, 0),
            new Combattant("Chevalier Squelette", 70, 18, 0),
            new Combattant("Dragon Noir (BOSS)", 110, 24, 0)
        };
        // Boucle FOR pour parcourir les 3 combats
        for (int i = 0; i < listeEnnemis.Length; i++)
        {
            Combattant ennemiActuel = listeEnnemis[i];
            Console.WriteLine("\n----------------------------------------");
            Console.WriteLine($"🔥 COMBAT {i + 1}/3 : Un {ennemiActuel.GetNom()} sauvage
apparait !");
            Console.WriteLine("----------------------------------------");
            // Boucle WHILE du combat
            while (heros.EstVivant() && ennemiActuel.EstVivant())
            {
                Console.WriteLine("\n--- Etat du combat ---");
                heros.AfficherBarreDeVie();
                ennemiActuel.AfficherBarreDeVie();
                int choixAction = 0;
                // Boucle DO...WHILE de saisie sécurisée
```

```
do
{
    Console.Write("\nQue veux-tu faire ? (1: Attaquer | 2: Boire une potion) : ");
    choixAction = int.Parse(Console.ReadLine());
} while (choixAction != 1 && choixAction != 2);
// SWITCH pour exécuter l'action
switch (choixAction)
{
    case 1:
        heros.Attaquer(ennemiActuel);
        break;
    case 2:
        heros.BoirePotion();
        break;
}
// Riposte de l'ennemi
if (ennemiActuel.EstVivant())
{
    ennemiActuel.Attaquer(heros);
}
else
{
```

```
                Console.WriteLine($"\n🏆 Victoire ! {ennemiActuel.GetNom()} a ete terrasse !");

            }

        }

        if (!heros.EstVivant())

        {

            Console.WriteLine($"\n💀 Tu es tombe au combat face a {ennemiActuel.GetNom()}...");

            break;

        }

    }

    // Bilan final

    Console.WriteLine("\n========================================");

    if (heros.EstVivant())

    {

        Console.WriteLine($"🎉 FELICITATIONS {heros.GetNom()} ! Tu es le grand Champion de l'Arene !");

    }

    else

    {

        Console.WriteLine("   GAME OVER - Retente ta chance !");

    }

    Console.WriteLine("========================================");
```

```
    }

}
```